

A Reproducible, Audit-Logged Container Pattern for Open-Source Intelligence Collection

Ali Murtaza Bhutto
ORCID: 0009-0007-2787-943X
alibhutto101112@gmail.com

Abstract—Open-source intelligence (OSINT) collection in production settings is typically performed by a fragmented toolchain that the operator assembles by hand on a workstation. The result is poor reproducibility, ad-hoc audit trails, and a non-trivial chance that artefacts from one engagement leak into the next. This paper describes *docker-osint-stack*, a Compose-defined deployment that bundles two widely used OSINT tools (SpiderFoot and theHarvester) with a PostgreSQL audit-log and an Nginx reverse proxy into a single-command launch, behind opinionated DevSecOps gates and an encrypted-export script. We report measured DevSecOps numbers (hadolint, Trivy, Dockle), measured runtime numbers (boot time, idle RAM, smoke-test outcomes), and discuss the contribution as a *packaging pattern* rather than a new detection technique. The artefact is open-source under MIT.

Index Terms—OSINT, Docker, container security, audit logging, reproducible deployment, DevSecOps, SpiderFoot, theHarvester

I. INTRODUCTION

OSINT collection has a tooling problem that is not, primarily, a detection problem. The community has converged on a small number of mature tools—SpiderFoot for aggregation across hundreds of passive sources, theHarvester for subdomain and email enumeration—and a sizeable body of practitioner literature on how to apply them [1]. What is generally *not* solved is the operational shell around them: an analyst working a real engagement needs (i) a reproducible build of every tool at a pinned version, (ii) a tamper-evident record of who ran what against whom and under what authorisation, (iii) a secure way to wipe and export the artefacts at engagement close, and (iv) a defensible security posture for the infrastructure that hosts the collection.

These needs are well understood in adjacent fields. The intelligence literature has long argued that the auditability of an analytic chain matters as much as the chain's content [2]. Container security guidance from NIST and CIS describes a clear set of practice gates—non-root users, no privilege escalation, healthchecks, minimal base images—that an OSINT deployment should meet just as any other production container would [3], [4]. Yet the tooling that most practitioners use does not ship in a form that satisfies these gates by default.

docker-osint-stack addresses the gap by composing existing tools, rather than building new ones, into an

environment that is trivial to stand up, hard to misuse, and *measured*—in the sense that every claim in its documentation is backed by a tool output that the reader can regenerate.

a) Contribution.: The contribution is a packaging pattern and the measurements that substantiate it, not a new detection capability. Specifically: (1) a Compose definition for a four-service stack (SpiderFoot, theHarvester, PostgreSQL audit-log, Nginx) with the externally reachable surface restricted to a single bound-to-localhost port; (2) an audit-log schema (Section III) that records sessions, tool invocations, normalised findings, and raw outputs in a way that survives the destruction of any single container's volumes; (3) a measured DevSecOps profile (Section V) that is honest about its non-zero CVE counts and explains the remediation posture; and (4) a workflow-template structure that forces an authorisation gate before any tool runs.

II. ARCHITECTURE

The stack consists of four services on a private Docker bridge network. Three are long-running (SpiderFoot, PostgreSQL, Nginx) and one is a one-shot CLI (theHarvester) gated behind a Compose profile so it does not start on `docker compose up`.

a) External surface.: The host port (default 127.0.0.1:8080) is the only externally reachable network surface. SpiderFoot's HTTP listener is bound only on the container network, with Nginx as the lone reverse proxy. This makes the operator-facing endpoint a single well-known location and prevents the common failure mode where an analyst accidentally exposes SpiderFoot's UI on a public IP.

b) Container hardening.: Every service inherits a YAML anchor that sets `no-new-privileges:true` and a restart policy of `unless-stopped`. Both custom Dockerfiles (SpiderFoot, theHarvester) use a two-stage build: a builder stage installs system packages and Python wheels, and a runtime stage copies only the required artefacts into a slim base image and runs as a dedicated non-root user (`spider` and `harvester` respectively). Each service that exposes a network port declares an HTTP or `pg_isready` healthcheck; `scripts/start.sh` polls these healthchecks with a three-minute deadline.

c) Persistence and teardown.: Two named volumes (`postgres-data`, `spiderfoot-data`) carry the state that

should survive a container restart. The `teardown.sh` script removes both by default; the `-shred` flag additionally calls GNU `shred` over the GPG-encrypted exports directory. The default-destructive behaviour is a deliberate choice: an engagement should end in known-clean state, and `-keep-volumes` exists for the cases where it should not.

III. AUDIT-LOG SCHEMA

The audit-log is a dedicated PostgreSQL 16 service so that destroying a SpiderFoot or theHarvester container does not destroy the engagement record. Schema initialisation is idempotent and loaded from `services/audit-log/init.sql` via the standard `/docker-entrypoint-initdb.d` mechanism in the official PostgreSQL image [5].

Four tables capture the audit trail:

- **investigation_sessions** — one row per engagement or question, carrying an `operator` handle and, crucially, an `authorisation` reference. A `CHECK` constraint refuses to record an empty authorisation, encoding the policy that the audit-log is not a free-form notepad.
- **tool_invocations** — one row per SpiderFoot scan or theHarvester run, with the tool’s arguments captured as JSONB so the row schema does not need to enumerate every possible flag.
- **findings** — normalised entities (subdomain, email, IP, hash, etc.) keyed to a `finding_severity` ENUM bucket (`info` through `critical`).
- **raw_outputs** — verbatim tool output as JSONB, foreign-keyed to the invocation. Storing the raw output alongside the normalised view lets later reanalysis run against the original artefact rather than against a destructive summary.

This structure mirrors the analytic argument [2] that intelligence work is only trustworthy when its provenance chain—raw, processed, analysed, disseminated—is itself inspectable. The schema makes that chain a query, not a footnote.

IV. WORKFLOWS AND AUTHORISATION GATE

Four investigation templates ship with the stack: *domain reconnaissance*, *social media OSINT* (organisation-centric), *credential exposure*, and a blank template. Each follows the same six-phase structure: scope and legal gate, Priority Intelligence Requirements (PIRs), theHarvester pass, SpiderFoot pass, triage table, and session close. The legal-gate phase is mandatory: the template begins with an explicit row for the authorisation reference, and the audit-log schema’s `CHECK` constraint enforces it at storage time as well.

The intent is to make it harder for a tool run to outrun its paperwork. This is a recurring failure mode in practitioner OSINT [1], and the most cost-effective place to address it is at the template, not at the tool.

TABLE I
TRIVY CVE COUNTS PER IMAGE (MEASURED).

Image	CRIT	HIGH	MED	LOW
spiderfoot:v4.0	8	46	120	118
harvester:4.10.1	3	17	72	94
postgres:16-alpine	1	13	19	1
nginx:1.27-alpine	6	29	46	9

TABLE II
DOCKLE FINDINGS PER IMAGE AFTER DOCUMENTED FALSE-POSITIVE FILTERS. THE SUPPRESSIONS ARE: `-af SETTINGS.PY` FOR PYTHON-DOCK (SPIDERFOOT) AND `PYTHON-SHODAN` (THEHARVESTER), BOTH LIBRARY FILES LITERALLY NAMED `SETTINGS.PY`; AND `-ak KEY_SHA512` FOR THE `NGINX:1.27-ALPINE` SIGNING-KEY CHECKSUM.

Image	FATAL	WARN
spiderfoot:v4.0	0	0
harvester:4.10.1	0	0
postgres:16-alpine	0	1
nginx:1.27-alpine	0	1

V. MEASURED POSTURE

All numbers in this section are reproducible by re-running the scripts in `tests/` and `results/`. They are not estimates.

A. Dockerfile lint (hadolint 2.12.0)

Both custom Dockerfiles pass hadolint with zero findings under the default ruleset.

B. CVE scan (Trivy 0.70.0)

Table I reports severity counts per image. The numbers are non-zero because SpiderFoot v4.0 (April 2022) pins dependency versions that have accreted CVEs since release. We do not suppress them; we report the patch path as *rebuild against the same tag* (which picks up upstream’s own dependency updates when they ship) and *rebase to a newer SpiderFoot tag* once one exists. The remediation posture treats CVE suppression as a worse failure mode than honest reporting, in keeping with both the NIST [3] and CIS [4] guidance for production container deployments.

C. Container best-practice (Dockle 0.4.15)

After two documented false-positive suppressions (see Table II caption), the custom images report zero FATAL findings; the official base images report one informational WARN each (“last user should not be root”), which is mitigated at Compose level by `no-new-privileges` and by both nginx and postgres themselves dropping to non-root worker processes after init.

D. Live boot

A cold `docker compose up` of the three long-running services reaches the “healthy” state, end-to-end, in 7.48 seconds on a commodity laptop (Linux, 8 cores, ≈ 10 GiB

free RAM at the time of measurement). The wall-clock figure comes from `/usr/bin/time -f '%e'` wrapping `scripts/start.sh`. Idle memory usage of the three services totals approximately 153 MiB (`dos-postgres` 36.9 MiB, `dos-spiderfoot` 106.2 MiB, `dos-nginx` 10.2 MiB).

E. Smoke test

The end-to-end smoke test (`tests/test_stack.sh`) runs ten assertions: Compose syntax, environment file presence, full stack boot, PostgreSQL readiness, SpiderFoot HTTP, Nginx `/healthz`, the Nginx-to-SpiderFoot proxy path, presence of the four audit-log tables, a CTE round-trip insert across `investigation_sessions` $\rightarrow$ `tool_invocations` $\rightarrow$ `findings`, and a theHarvester help invocation via the one-shot profile. All ten pass on the reference host.

VI. THREAT MODEL AND LIMITATIONS

We are explicit about what the stack does *not* defend against:

- **TLS at the edge.** The default deployment terminates plain HTTP on 127.0.0.1. Externalising the UI requires a TLS-terminating reverse proxy and authentication in front. We do not bundle either, because the choice (Caddy vs. Traefik vs. nginx with cert-manager, basic-auth vs. OIDC) is engagement-specific.
- **Compromised upstream.** We pin to tags, not commit hashes, so a rebuild trusts whatever upstream has retroactively pushed behind a tag. The mitigation is to fix a commit hash in regulated environments. The default pins are easier to keep current; the more conservative pins are more reproducible.
- **Per-user authentication.** SpiderFoot’s UI is single-tenant. The stack assumes one trusted operator per host.

These limitations are visible in the public README; they are not hidden behind a marketing claim of generality.

VII. CONCLUSION

`docker-osint-stack` is deliberately small. It does not propose a new OSINT methodology; the methodology is in the workflow templates and rests on existing practitioner work [1], [2]. It does not propose a new container-security regime; the regime is the existing NIST [3] and CIS [4] guidance, applied honestly. What it contributes is a packaging that makes the combination *operable in a single command* and a measurement discipline that makes every claim in its documentation auditable. The artefact is MIT-licensed and intended for authorised security engagements only; the `ETHICAL_USE.md` file enumerates the out-of-scope cases.

REFERENCES

- [1] M. Bazzell, *Open Source Intelligence Techniques: Resources for Searching and Analyzing Online Information*, 10th–11th eds., IntelTechniques.com, 2023–2024.
- [2] R. J. Heuer, *Psychology of Intelligence Analysis*, Center for the Study of Intelligence, Central Intelligence Agency, 1999.
- [3] M. Souppaya, J. Morello, and K. Scarfone, “Application Container Security Guide,” National Institute of Standards and Technology, NIST Special Publication 800-190, Sep. 2017.
- [4] Center for Internet Security, “CIS Docker Benchmark,” CIS, Tech. Rep., latest version, accessed 2026. <https://www.cisecurity.org/benchmark/docker>
- [5] PostgreSQL Global Development Group, “PostgreSQL Documentation,” v16, 2024–2025. <https://www.postgresql.org/docs/16/index.html>